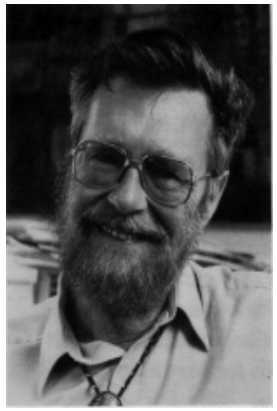
Anweisungsarten und deren Semantik

Imperative Programmierung: Anweisungen

- Bausteine aus denen Programme (Algorithmen) aufgebaut werden
- Anweisungsarten in der ***strukturierten*** prozeduralen ***Programmierung***:
  - ***Zuweisung***
  - ***Sequenz***
  - ***Alternative (Bedingte Verzweigung): if, switch***
  - ***Wiederholung (Schleife): while, do, for***
  - ***Block von Anweisungen***



E.W. Dijkstra

- alle Anweisungen sollten einem Prinzip folgen:
  - **„single entry & single exit“**
  - jede Anweisung hat einen wohldefinierten Start- und Endpunkt
    - z.B. ein Sprung in die oder aus der Mitte eines Anweisungsblocks ist nicht möglich
  - Sprünge (goto) sollten von höheren Programmiersprachen gemieden werden
    - können durch strukturierte Anweisungen vollkommen ersetzt werden
- beliebige Kombination der Grundformen ist möglich - ***„Schachtelung“***
  - haben ebenfalls „single entry & single exit“-Eigenschaft
  - erlauben Umsetzung komplexer Algorithmen (Turing-Vollständigkeit)

Zuweisung und Sequenz: Allgemeines

- in Programmiersprachen gibt es neben der einfachen Zuweisung oft die ***Mehrfachzuweisung***

  a = a + c; a = b = c = 0; a=(b=(c=0)));

  - Mehrfachzuweisungen werden von rechts nach links ausgewertet („rechtsassoziativ“)

- können Zuweisungen auch als reine Anweisungen vorkommen (und deshalb nicht in Ausdrücken verwendet werden)

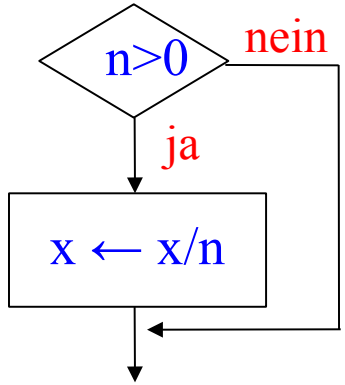
- können Zuweisungen eine ***Kopier- oder Referenzsemantik*** verfolgen

- Sequenz steht für eine nacheinanderfolgende Abarbeitung von ein oder mehreren, beliebig geschachtelten Anweisungen $stmt_1$, ..., $stmt_n$

  $stmt_1$; ... ;$stmt_n$;

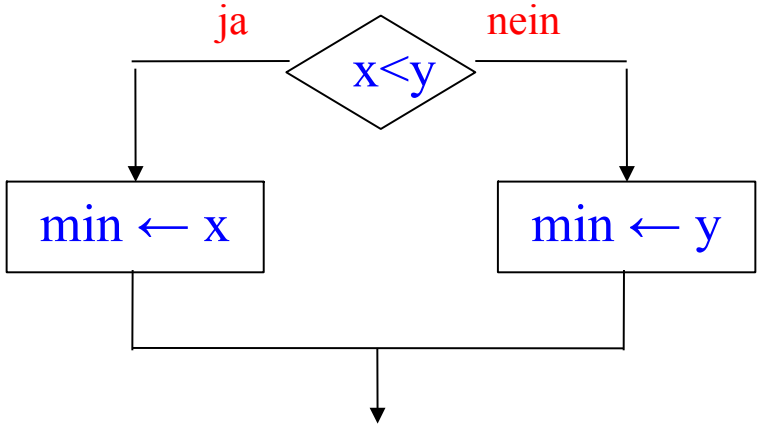
Verzweigungen

If-Anweisung




```
if (n > 0)
    x = x/n;
```

ohne else-Zweig




```
if (x< y)
    min = x;
else
    min = y;
```

mit else-Zweig

Syntax

IfStatement = "if" "(" Expression ")" Statement ["else" Statement]

Anweisungsblöcke

wenn if- oder else-Zweig aus mehr als einer Anweisung bestehen, müssen sie geklammert werden.

```
Statement = Assignment | IfStatement | ... | Block.
Block = "{" {Statement} "}".
```

Beispiel:

```
if ( x<0) {
    negNumbers++;
    printf("%d \n", x);
} else {
    posNumbers++;
    printf("%d \n", x);
}
```

Anweisungsblöcke

- Anweisungen in Blöcken werden sequentiell abgearbeitet
- in C können in Blöcken vereinbarte Variablen die Variablen übergeordneter Blöcke überdecken
- im Standard-C90 dürfen Deklarationen nur am Anfang des Blocks stehen, ab Standard-C99 aber gemischt deklariert werden

```
int x;
scanf("%d", &x);
if (x < 0) {
    int y;   // Gültige Variablendefinition
    int x;   // Gültige Variablendefinition
    y = 5;
}
```

Einrückungen

- erhöhen die Lesbarkeit (machen Programmstruktur besser sichtbar)
- Einrückungstiefe: ~~1 Tabulator~~ oder 2 Leerzeichen

```
if (n > 0)
    x = x/n;

if(x < y)
    min = x;
else
    min = y;

if (x < 0){
    negNumbers++; printf("%d \n", x);
} else {
    posNumbers++; printf("%d \n", x);
}
```

- kurze if-Anweisungen können auch in eine Zeile geschrieben werden

```
if(n > 0) x = x/n;                    // ok, aber nicht gut
if(x < y) min = x; else min = y;      // richtig schlecht
```

If-Anweisungen: Dangling Else

```
if (x >= 0)
     if ( x > 0)
          printf("größer als Null \n");
else
       printf("kleiner als Null \n");
```

```
if (x >= 0)
  if ( x > 0)
       printf("größer als Null \n");
  else
       printf("kleiner als Null \n");
```

Mehrdeutigkeit! Zu welchem if gehört das else?

Regel: ***else gehört immer zum unmittelbar vorausgehenden if***

Wenn man das ***nicht wollte***, muss man die Anweisung so schreiben

```
if (x >= 0){
     if ( x > 0)
          printf("größer als Null \n");
}else
     printf("kleiner als Null \n");
```

Datentypen der ganzen Zahlen: Vergleichsoperatoren

Vergleich zweier Werte liefert wahr (true) oder falsch (false)

	Bedeutung	Beispiel
==	gleich	x == 3
!=	ungleich	x != y
>	größer	4 > 3
<	kleiner	x+1 < 0
>=	größer oder gleich	x >= y
<=	kleiner oder gleich	x <= y

Beispiel:

```
if (x == 0) printf("x  is zero \n");
```

Achtung: "=" ist in C kein Vergleich, sondern Zuweisungsoperator

```
if (x = 0) printf("x is zero \n");      // Compiler meldet keinen Fehler
If (0 = x) printf("x is zero \n");      // vermeidet dieses Problem
```

Der Datentyp: boolean

Der Datentyp boolean: Allgemeine Betrachtung

Bedingung einer If-Anweisung muss einen Wahrheitswert liefern

Datentyp ***boolean*** repräsentiert die Wahrheitswerte true und false zulässige Operationen sind

Operator	Bedeutung
&	und - mit vollständiger Auswertung
|	oder - mit vollständiger Auswertung
^	wahr, wenn Operanden unterschiedlich sind
!	Negation
&&	und - mit Kurzschlussauswertung
||	oder - mit Kurzschlussauswertung

Beispiele:

```
x > 0
p || q && x < 10
```

Aber: C hat keinen Datentyp boolean !!
Oder doch? Dazu später mehr.

Boolesche Operatoren: Allgemeine Betrachtung

und-, oder- und nicht-Verknüpfungen

x	y	x & y, x && y
true	true	true
true	false	false
false	true	false
false	false	false

x	y	x | y, x || y
true	true	true
true	false	true
false	true	true
false	false	false

x	!x
false	true
true	false

Beispiel

```
if( x >= 0 & x <= 10 | x >= 100 & x <= 110) y = x;
```

- **Vorrangregeln**

  ! bindet stärker als &
  & bindet stärker als |
  | bindet stärker als &&
  && bindet stärker als ||

- **Vorrangregeln können durch Klammerung umgangen werden:**

  ```
  if (x > 0 & (y==1 || y==7)) ....
  ```

Kurzschlussauswertung: Allgemeine Betrachtung

Zusammengesetzter Vergleich wird abgebrochen, sobald Ergebnis feststeht

äquivalent!

```
if (a != 0 && b/a > 0) x = 0;
```

wenn false, dann ist gesamter Ausdruck false

```
if (a != 0)
    if (b/a > 0)
        x = 0;
```

```
if (a == 0 || b/a > 0)   x = 1;
```

wenn true, dann ist gesamter Ausdruck true

```
if (a == 0)
    x = 1;
else if (b/a > 0)
    x = 1;
```

Achtung: Semantik einer Kurzschlussauswertung kann verschiedenartig zu der einer vollständigen Auswertung sein

Kurzschlussauswertung: Verschiedenartige Semantik

```
int x = 1, y = 1, w = 10, h = 10;
if( (w == h) || (x++ > y))              // keine vollständige Auswertung
      printf("x = %d \n", x);
// Ausgabe: x = 1
```

aber!

```
int x = 1, y = 1, w = 10, h = 10;
if( (w == h) | (x++ > y))               // vollständige Auswertung
      printf("x = %d \n", x);
// Ausgabe: x = 2
```

Achtung: **In C gibt es keinen Datentyp boolean. Boolesche Werte werden durch den Datentyp int repräsentiert.**

Datentyp boolean: Realisierung in C

- C89 und früher kennt keinen eigenen Datentyp boolean. Anstatt der Wahrheitswerte true und false werden Zahlen verwendet
  - 0 steht für falsch, jede Zahl ungleich 0 für wahr
- Operationen &&, ||, ! des Datentyps boolean sind in C dann als Operationen auf ganzen Zahlen gegeben
  - liefern 0 für falsch und 1 für true

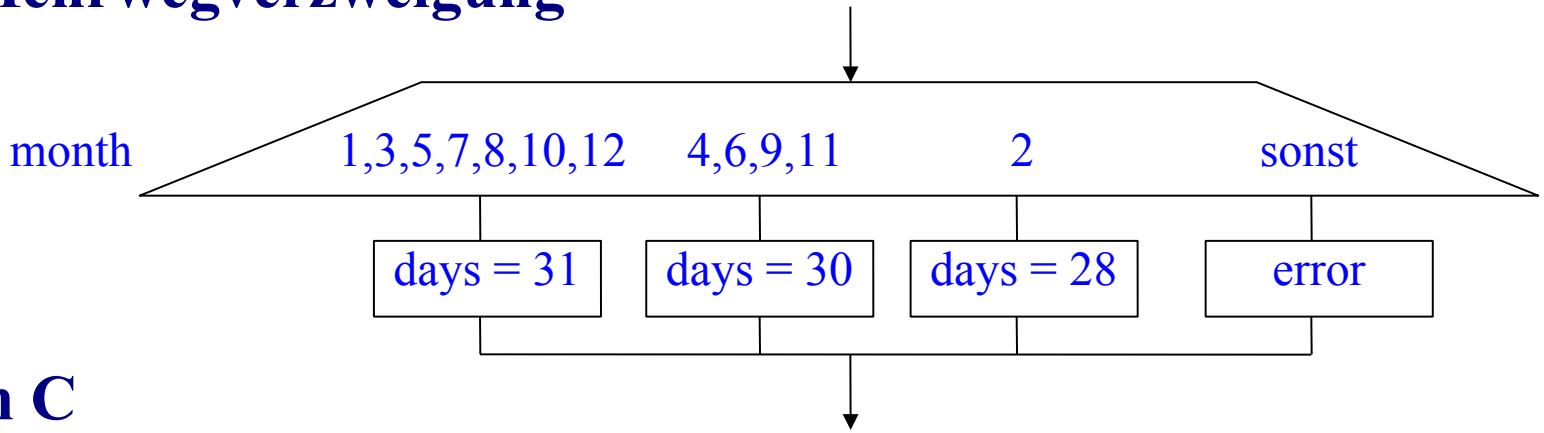
```
0 && 1      // Ergebnis 0
0 || 1      // Ergebnis 1
```

- Operationen && und || unterliegen der Kurzschlussauswertung
- Ausführung der Operanden bei &&, || ist fest (von links nach rechts)
- Achtung: & und | stehen in C nicht für boolesche Operationen, sondern für bitweises AND und OR auf ganzen Zahlen
- C99 führt Datentyp _Bool ein, sowie Makros zur Definition von bool, true und false in <stdbool.h>

Mehrwegverzweigung

Switch-Anweisung

Mehrwegverzweigung



in C

```
switch (month){
        case 1: case 3: case 5: case 7: case 8: case 10: case 12:
                days = 31; break;
        case 4: case 6: case 9: case 11:
                days = 30; break;
        case 2:
                days = 28; break;
        default:
                printf("error \n");
}
```

Switch-Anweisung: Semantik

Selektor-Ausdruck

```
switch (month){
      case 1: case 3: case 5: case 7:
      case 8: case 10: case 12:
            days = 31; break;
      case 4: case 6: case 9: case 11:
            days = 30; break;
      case 2:
            days = 28; break;
      default:
            printf("error \n");
}
```

Break-Anweisung

- springt ans Ende der Switch-Anweisung
- wenn break fehlt, läuft Programm über nächste case-Marke weiter (häufige Fehlerursache)

Semantik

1. berechne Selektor-Ausdruck
2. springe zur passenden case-Marke
   - wenn keine passt springe zu default
   - wenn kein default angegeben ist, springe ans Ende der Switch-Anweisung

Bedingungen

1. Selektor-Ausdruck muss ganzzahlig oder char sein
2. case-Marken müssen Konstanten sein
3. ihr Typ muss zum Typ des Selektor-Ausdrucks passen
4. case-Marken müssen voneinander verschieden sein

Switch-Anweisung: Beispiel

```
int x = 4;
switch(x){
        case 3: printf("Drei \n");
        case 4: printf("Vier \n");
        case 5: printf("Fünf \n");
        default: printf("was anderes \n");
    }
```

Ausgabe:

```
Vier
Fünf
was anderes
```

Switch-Anweisung: Beispiel

```
int x = 4;
switch(x){
        case 3: printf("Drei \n"); break;
        case 4: printf("Vier \n"); break
        case 5: printf("Fünf \n"); break;
        default: printf("was anderes \n"); break;
    }
```

Ausgabe:

Vier

Switch-Anweisung: Syntax

Statement = Assignment | IfStatement | SwitchStatement | ... | Block.

SwitchStatement = "switch" "(" Expression ")" "{" {Labels Statements} "}".

Labels = Label {Label}.

Statements = Statement {Statement}.

Label = "case" ConstantExpression ":" | "default" ":".

Bemerkung: In case-Fällen können im Standard-C99 auch Variablen definiert werden. Diese sind in der gesamten switch-Anweisung sichtbar, aber nur im definierenden case-Fall initialisiert.

Unterschied zwischen If und Switch

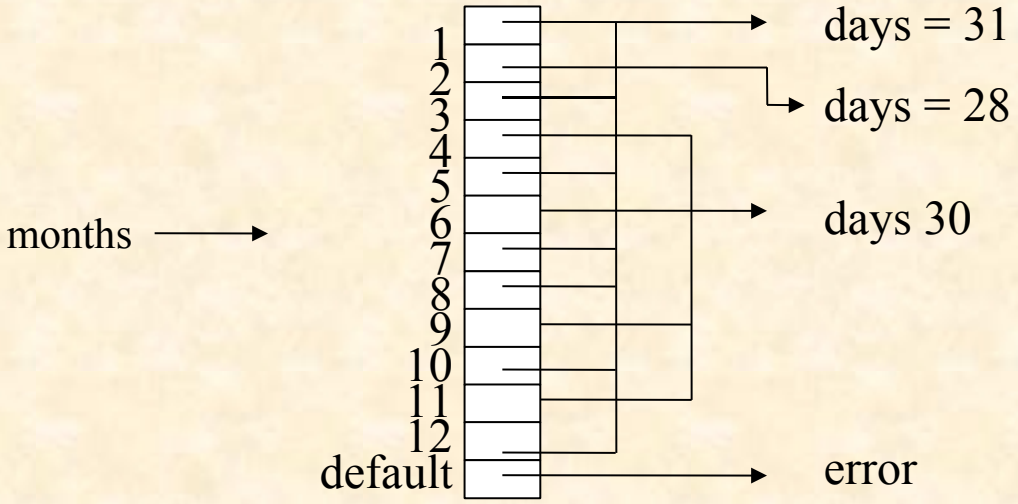
```
if( month == 1 || month == 3 || month == 5 ||
    month == 7 || month == 8 || month == 10 ||
    month == 12)
        days = 31;
else if (month ==  4 || month == 6 || month == 9 ||
        month == 11)
        days = 30;
else if (month == 2)

        days = 28;
else
        printf("error \n");
```

prüft Bedingungen sequentiell

```
switch (month){
        case 1: case 3: case 5: case 7:
        case 8: case 10: case 12:
           days = 31; break;
        case 4: case 6: case 9: case 11:
           days = 30; break;
        case 2:
           days = 28; break;
        default:
        printf("error \n");
}
```

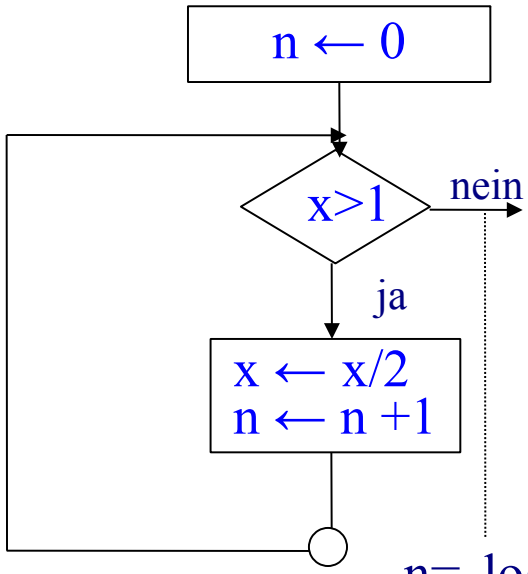
benutzt Sprungtabelle!



Schleifen

While-Schleife (Abweisende Schleife)

Führt eine Anweisungsfolge aus, solange eine bestimmte Bedingung gilt




```
int n = 0;
while (x > 1){
      x = x/2;
      n = n+1;
}
```

Schleifenbedingung

Schleifenrumpf

Syntax

Statement = Assignment | IfStatement | SwitchStatement | WhileStatement | ... | Block.

WhileStatement = "while" "(" Expression ")" Statement.

Wenn Schleifenrumpf aus mehreren Anweisungen besteht, muß er mit {...} geklammert werden

Beispiel: Summenberechnung

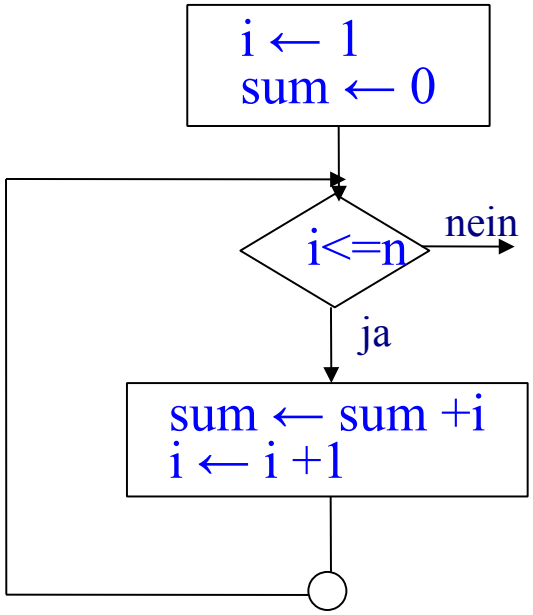
Aufgabe: Summe der natürlichen Zahlen von 1 bis n berechnen

```
/* datei: summe.c */

#include <stdio.h>

    int main(void){
            int n; int i; int sum;

            scanf("%d ", &n);
            i = 1;
            sum = 0;
            while (i <= n){
                sum = sum + i;
                i = i + 1;
            }
            printf("Sum %d \n ", sum);
            return 0;
    }
```



Beispiel: Histogramm-Ausgabe

Aufgabe: Zahlenfolge lesen und Histogramm ausgeben

Eingabe: 3 2 5 Ausgabe: *** ** *****

```
/* Datei histogramm.c */
 #include <stdio.h>
 int main(void){
      int i;
     scanf(" %d", &i);
     while (i >= 0){


              // verarbeite Zeichen
              scanf(" %d", &i);
     }
}
```

liest die Zahlenfolge

Beispiel: Histogramm-Ausgabe

Aufgabe: Zahlenfolge lesen und Histogramm ausgeben

Eingabe: 3 2 5 Ausgabe:

```
***
**
*****
```

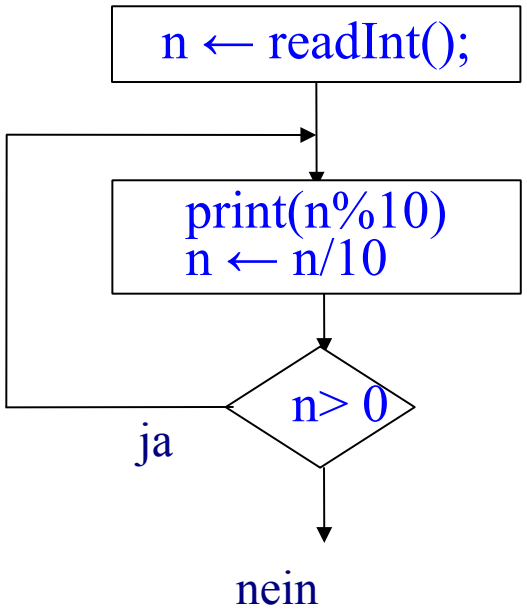
```
/* Datei histogramm.c */
 #include <stdio.h>
 int main(void){
      int i;
      scanf(" %d", &i);
      while (i >= 0){
          int j = 1;
          while (j <= i) {printf("*"); j++;}
          printf("\n");
          scanf(" %d", &i);
      }
}
```

liest die Zahlenfolge

gibt i Sterne aus

Do-While-Schleife (Annehmende Schleife)

Abbruchbedingung wird am Ende der Schleife geprüft




```
int n;
scanf("%d', &n);
do {
      printf("%d", n%10);
      n = n/10;
} while (n > 0);
```

Syntax

Statement = Assignment | IfStatement | SwitchStatement | WhileStatement | DoWhileStatement | ... | Block.

DoWhileStatement = "do" Statement "while" "(" Expression ")" ";".

Wenn Schleifenrumpf aus mehreren Anweisungen besteht, muß er mit {...} geklammert werden

For-Schleife

Allgemein eine Zählschleife, die eingesetzt werden sollte, wenn Anzahl an Schleifendurchläufen im voraus bekannt ist

Allgemeine Form:

for(Initialisierung; Test; Inkrementierung)
Anweisung(en);

Beispiel

```
int i;
int sum = 0;
for(i = 1; i <= n; i++)
      sum = sum + i;
```

Kurzform für

```
int i;
int sum = 0;
i = 1;                  // Initialisierung
while (i <= n){  // Test
      sum = sum + i;
      i++;              // Inkrementierung
}
```

Syntax der For-Schleife

ForStatement = "for" "(" [ForInit] ";" [Expression] ";" [ForUpdate] ")" Statement.

ForInit = Assignment {"," Assignment}

| Type VarDecl {"," VarDecl}

ForUpdate = Assignment {',' Assignment}

Beispiele:

```
for (i =0; i < n; i++) ...
for (i=10; i > 0; i--) ...
for (int i = 0; i <= n; i = i + 1) ...
for (int i = 0, j = 0; i > 0 && j < n; i = i + 1, j = j+1) ...
for (;;) ...
```

ein oder mehrere Variablen können in Standard-C99 im Initialisierungsteil der for-Schleife definiert werden

Beispiel: Multiplikationstabelle drucken

```
/* Datei: printtable.c */
#include <stdio.h>
        int main(void){
                int n;
                scanf("%d", &n);
                for (int i = 1; i <= n; i++){
                        for (int j = 1; j <= n; j++)
                                printf("%d ", i * j);
                        printf("\n");
                }
                return 0;
        }
```

Ausgabe:

```
    1  2  3
  +---------
1 |  1  2  3
2 |  2  4  6
3 |  3  6  9
```

innere Schleife gibt jeweils die i-te Zeile aus

Hochsprachliche Sprunganweisungen

Sprunganweisung break

Verlassen von Schleifen mit der break-Anweisung

Beispiel 1:

```
while (true){
        . . .
        if (++i == 5) break;
}
```

Beispiel 2:

```
char eingabe;
 for (; ;){
      eingabe = getchar();
      if(eingabe == 'Q') break;
        …..
}
```

break hat nur eine Auswirkung auf die innere Schleife

Fortsetzung einer Schleife mit continue

durch die Anweisung continue wird der aktuelle Schleifendurchlauf unterbrochen

```
int y = 0;
while ( y < 2 ){
      y++;
      int x = 0;
      while ( x <= 3 ) {
                  if ( ++x <= 3 ) continue;
                  printf("x = %d y = %d ", x, y);
      }
}
```

Ausgabe:

x = 4 , y = 1
x = 4 , y = 2

continue hat nur eine Auswirkung auf die innere Schleife

Direkte Sprünge mit goto-Sprunganweisungen

Sprunganweisung: goto und Marken

mit goto-Anweisungen kann im Programm nach beliebigen Programmpunkten gesprungen werden

```
while (….){
      while (….) {
            ….
            if ( Abbruch ) {
                  goto Weiter_nach_Schleife;
            }
      }
     ….
}
Weiter_nach_Schleife:
Anweisungen
```

Aber: goto-Sprunganweisungen haben in gut strukturierten Programmen nichts zu suchen